PROGRAMACIÓN EN CIENCIA DE DATOS

REPORTE DE ACTIVIDAD PRÁCTICA

GUÍA INTEGRAL DE INSTALACIÓN DE SHINY, CONSTRUCCIÓN DE UN DASHBOARD BÁSICO Y DESPLIEGUE EN POSIT

Proceso Paso a Paso: Desde la Instalación de Librerías hasta la Inicialización y Publicación en la Nube

Actividad:	Instalación de Shiny, Dashboard Básico y Publicación en Internet
Cuenta en Posit:	rncm11 (https://connect.posit.cloud/rncm11)
Repositorio GitHub:	https://github.com/RolandoCastillo11/Shiny-Prog-CD
Herramientas:	Python 3.11, Shiny for Python (v1.8.0), Posit Connect Cloud, Git
Fecha:	Octubre de 2026

1. Introducción y Requisitos Previos

El propósito de esta práctica es familiarizarse con el framework **Shiny for Python**, instalando su entorno completo, estructurando un **dashboard básico e interactivo**, documentando su inicialización y publicándolo en internet mediante **Posit Connect Cloud** (cuenta: rncm11) y **GitHub**.

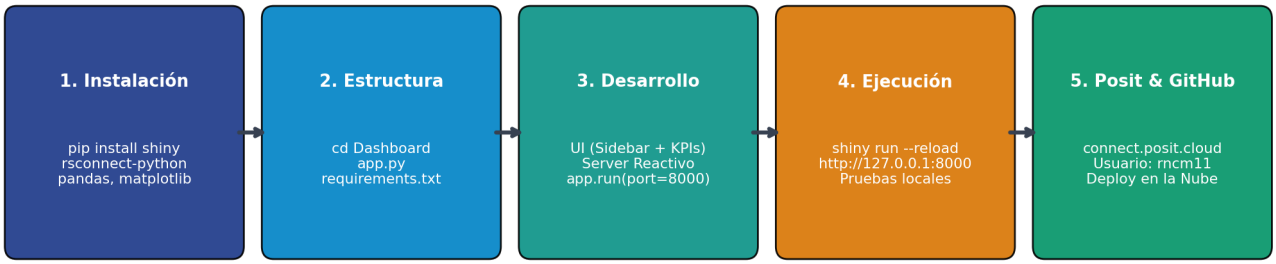


Figura 1: Fases secuenciales de la actividad práctica.

2. Paso a Paso: Instalación de Shiny y Dependencias

Se realizaron los siguientes pasos en la consola Windows PowerShell:

1. **Verificación de Python:** python --version (Python 3.11.0 detectado).
2. **Instalación de Shiny:** pip install shiny (instala el servidor ASGI Uvicorn, Starlette, HTMLTools y BSLib).
3. **Instalación de Librerías Analíticas:** pip install numpy pandas matplotlib jinja2 rsconnect-python.
4. **Comprobación de Versiones:** shiny --version (retorna versión 1.8.0) y rsconnect version (retorna 1.32.0).

3. Preparación del Directorio y Desarrollo del Dashboard Básico

Se creó la carpeta Dashboard_Basico_Shiny/ para alojar app.py, requirements.txt y el control de versiones. El dashboard básico explora distribuciones estadísticas con entradas, KPIs, gráfica y tabla:

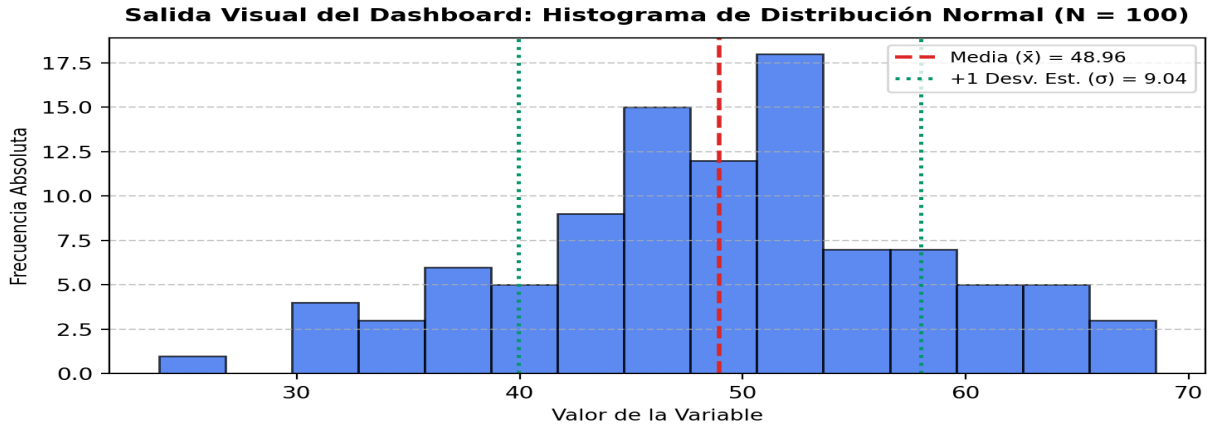


Figura 2: Histograma de frecuencias generado reactivamente por el Dashboard.

Código fuente representativo de app.py:

```
from shiny import App, render, ui, reactive
import numpy as np, pandas as pd, matplotlib.pyplot as plt

# --- 1. Interfaz de Usuario (UI) ---
app_ui = ui.page_sidebar(
    ui.sidebar(
        ui.input_slider('n_muestras', 'Muestras (N):', min=20, max=500, value=100, step=10),
        ui.input_select('tipo_distribucion', 'Distribución:', {'normal':'Normal','uniforme':'Uniforme','
exponencial':'Exponencial'}),
        ui.input_radio_buttons('color_grafica', 'Color:', {'#2563EB':'Azul','#059669':'Verde','#DC2626':
'Rojo'}),
        width=280,
    ),
    ui.h3('Dashboard Básico: Generador de Distribuciones Estadísticas'),
    ui.layout_columns(
        ui.value_box('Media (x■)', ui.output_text('kpi_media'), theme='primary'),
        ui.value_box('Desv. Est. (σ)', ui.output_text('kpi_desviacion'), theme='success'),
        ui.value_box('Total Muestras', ui.output_text('kpi_total'), theme='info'),
        col_widths=[4, 4, 4],
    ),
    ui.layout_columns(
        ui.card(ui.card_header('Histograma'), ui.output_plot('grafico_histograma')),
        ui.card(ui.card_header('Primeras 10 Observaciones'), ui.output_data_frame('tabla_primeros_datos'
)),
        col_widths=[7, 5],
    ),
    title='Dashboard Básico - Shiny',
)

# --- 2. Lógica del Servidor (Server) ---
def server(input, output, session):
    @reactive.calc
    def generar_datos():
        n, tipo = input.n_muestras(), input.tipo_distribucion()
        np.random.seed(42)
        if tipo == 'normal': vals = np.random.normal(50, 10, n)
        elif tipo == 'uniforme': vals = np.random.uniform(20, 80, n)
        else: vals = np.random.exponential(20, n)
        return pd.DataFrame({'ID': range(1, n+1), 'Valor': np.round(vals, 2)})

    @render.text
    def kpi_media(): return f"{generar_datos()['Valor'].mean():.2f}"
    @render.text
    def kpi_desviacion(): return f"{generar_datos()['Valor'].std():.2f}"
    @render.text
    def kpi_total(): return f"{len(generar_datos())} datos"

    @render.plot
    def grafico_histograma():
        fig, ax = plt.subplots(figsize=(6, 4))
        ax.hist(generar_datos()['Valor'], bins=15, color=input.color_grafica(), edgecolor='black')
        return fig

    @render.data_frame
    def tabla_primeros_datos(): return render.DataGrid(generar_datos().head(10))

# --- 3. Inicialización ---
app = App(app_ui, server)
if __name__ == '__main__':
    app.run(port=8000)
```

4. Cómo se Inicializa y Ejecuta la Aplicación

Para correr la aplicación es indispensable ubicarse en el directorio del script:
cd "Dashboard_Basico_Shiny"
Opción A (Servidor Shiny con recarga en caliente): shiny run --reload --port 8000 app.py
Opción B (Ejecución directa en Python): python app.py
La consola confirma: *Uvicorn running on http://127.0.0.1:8000*. Al abrir esa URL en el navegador se interactúa con el dashboard.

5. Resolución de Incidencias Durante la Práctica

• **Error '.\app.py not found':** Ocurre si se ejecuta el comando fuera de la carpeta Dashboard_Basico_Shiny. Solución: cd Dashboard_Basico_Shiny.
• **Error 'Import Jinja2 failed':** Ocurría por @render.table llamando a DataFrame.style. Solución: Se migró a @render.data_frame con render.DataGrid (nativo de Shiny) y se añadió jinja2>=3.1.0 a requirements.txt.

6. Tramitación de Cuenta en Posit y Publicación a Internet

• **Cuenta en Posit Connect Cloud:** https://connect.posit.cloud/rncm11 (Usuario: rncm11).
• **Repositorio en GitHub:** https://github.com/RolandoCastillo11/Shiny-Prog-CD.
• **Publicación en la Nube:** En *connect.posit.cloud* se pulsa **Publish** -> **GitHub** -> **RolandoCastillo11/Shiny-Prog-CD**. Posit detecta app.py y requirements.txt, compila el contenedor seguro y entrega el enlace público HTTPS.

7. Tabla de Verificación de la Práctica

Actividad	Herramienta / Evidencia	Estado
1. Instalación de Shiny	pip install shiny (v1.8.0)	✓ **Completada**
2. Dashboard Básico	app.py (Inputs, KPIs, Histograma)	✓ **Completado**
3. Inicialización Local	shiny run / python app.py (:8000)	✓ **En Ejecución**
4. Cuenta en Posit	connect.posit.cloud/rncm11	✓ **Tramitada**
5. Repositorio GitHub	Shiny-Prog-CD	✓ **Sincronizado**